PERTEMUAN 15 · Praktik terbimbing · Penutup proyek

AST, Validasi Kode, dan Penanganan Kesalahan

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 04 — Mengimplementasikan parser sederhana untuk validasi kode; CPMK 01 — Menjelaskan tahapan kompilasi.

Sub-CPMK yang dinilai

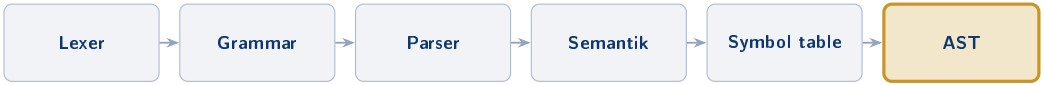
- **Sub-CPMK 4.4:** Membuat AST dan validator kode untuk subset bahasa C, lalu menguji keluaran pesan kesalahan sintaksis yang informatif.
- **Sub-CPMK 1.7:** Menguji dan men-*debug* penanganan kesalahan semantik sehingga program menghasilkan pesan kesalahan yang menyebut baris, nama, dan jenis pelanggaran.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
20	Pembuka: posisi M5 dan cakupan proyek akhir
25	Ceramah: desain AST dan traversal
30	Praktik: validator dan aturan validasi
30	Praktik: diagnostik dan pengujian pesan
25	Pengujian akhir pipeline dan demo
20	Laporan, refleksi, dan penutup proyek

Porsi praktik 85 menit. Pertemuan ini menutup increment proyek akhir (M5) sebelum UAS.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini menutup simpul **AST** dan menyatukan seluruh rantai menjadi validator.
- Pipeline lengkap: lexer, parser, type checker, validator, diagnostik.
- Keluaran akhir adalah CLI `scc` untuk Subset-C.

Prinsip Desain AST

1. Setiap simpul merepresentasikan satu konstruksi bahasa.
2. Informasi sintaksis yang tidak bermakna dihilangkan.
3. Simpul menyimpan lokasi untuk diagnostik.
4. Struktur mendukung traversal yang mudah.
5. Tipe dan informasi semantik dapat ditambahkan tanpa mengubah bentuk dasar.

Generik versus spesifik

Simpul generik mudah dipelajari, tetapi kurang *type-safe*. Simpul spesifik memberi kejelasan dan mencegah akses field yang salah; untuk proyek besar biasanya lebih baik.

Struktur Simpul AST (ast.h)

```
37 typedef struct AstNode {
38     AstKind kind;
39     Token token;
40     TypeKind decl_type;
41     struct AstNode *left;
42     struct AstNode *right;
43     struct AstNode *children;
44     struct AstNode *next;
45 } AstNode;
46
47 AstNode *ast_new(AstKind kind, Token token);
48 void ast_add_child(AstNode *parent, AstNode *child);
49 const char *ast_kind_name(AstKind kind);
50 void ast_print(FILE *out, const AstNode *node, int
       indent);
51 void ast_free(AstNode *node);
```

Listing 1: Struktur AstNode dan antarmukanya

- kind: jenis simpul.
- token: lokasi dan lexeme.
- decl_type: tipe deklarasi.
- left/right: operand.
- children/next: daftar anak.

Konstruksi AST

- Aturan yang hanya mengatur sintaksis tidak membuat simpul.
- Aturan operator membuat simpul biner dengan dua anak.
- Aturan daftar membuat simpul dan menambahkan anak.
- Deklarasi menyimpan tipe pada `decl_type`.
- AST dibangun saat parsing, dari bawah ke atas.

Dump AST untuk `int a = 2 + 3;`

`PROGRAM > FUNC main > BLOCK > DECL a > BINARY + > INT_LIT 2, INT_LIT 3.`
Tidak ada titik koma atau kurung pada AST.

Konstruksi Simpul: `mknode` dan `mknode`

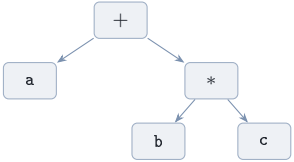
- Juarna memakai dua operasi dasar: `mkleaf` untuk daun dan `mknode` untuk simpul operator.
- `mkleaf(id, a)`, `mkleaf(id, b)`, `mkleaf(id, c)`.
- `mknode(*, b, c)` menghasilkan n_1.
- `mknode(+, a, n_1)` menghasilkan akar n_2.
- Di C, keduanya dibungkus menjadi `ast_new`.

Hasil untuk `a + b * c`

Akar `+`; anak kiri `a`; anak kanan `*`; cucu `b` dan `c`.

Traversal AST

- **Pre-order**: simpul lalu anak-anak.
- Cocok untuk analisis yang butuh konteks orang tua, misalnya validasi loop.
- **Post-order**: anak-anak lalu simpul.
- Cocok untuk sintesis, misalnya perhitungan tipe.



Pre-order: $+, a, *, b, c$

Pola *Visitor* dan Konteks

- *Visitor* memisahkan operasi dari struktur data.
- Implementasi C: fungsi yang menerima simpul dan konteks.
- Proyek Subset-C memakai `switch` pada `kind`.
- Setiap operasi baru menjadi fungsi traversal terpisah: `typecheck`, `validate`.
- `visit_children` menghindari duplikasi kode traversal.

Konteks traversal

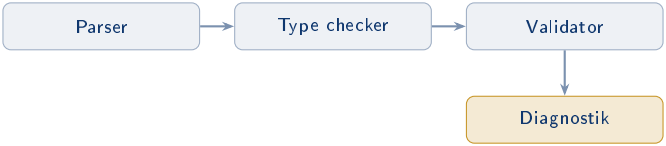
Berisi `loop_depth`, `return_type`, dan `has_return`. Konteks diperbarui saat masuk atau keluar simpul.

Aturan Validasi Kode

1. `break` dan `continue` hanya di dalam loop.
2. Fungsi non-`void` harus mengembalikan nilai.
3. `main` harus `int main()` tanpa parameter.
4. `return` hanya di dalam fungsi.
5. Pernyataan setelah `return` tidak terjangkau.
6. Pemanggilan fungsi sesuai *signature*.

Setiap aturan memiliki pesan yang jelas dan lokasi yang tepat.

Validasi Multi-Pass



1. Parsing menghasilkan AST.
2. Analisis semantik membangun tabel simbol dan memeriksa tipe.
3. Validasi memeriksa aturan kontrol dan kelengkapan.
4. Pelaporan mengumpulkan dan mengurutkan diagnostik.

Validator: Fungsi `visit`

```
31 static void visit(Context *context, AstNode *node) {
32     if (!node) {
33         return;
34     }
35
36     switch (node->kind) {
37     case AST_FUNC: {
38         Context inner;
39         inner.loop_depth = 0;
40         inner.return_type = node->decl_type;
41         inner.has_return = 0;
42         inner.had_error = 0;
43
44         visit_children(&inner, node);
```

Listing 2: Awal fungsi `visit` pada `validate.c`

- Simpul fungsi memulai konteks baru.
- `return_type` dan `loop_depth` diatur.
- Anak dikunjungi dengan konteks tersebut.
- Fungsi non-void tanpa `return` dilaporkan.

Titik Masuk Validator

```
92  int validate_program(AstNode *root) {
93      Context context;
94      context.loop_depth = 0;
95      context.return_type = TYPE_VOID;
96      context.has_return = 0;
97      context.had_error = 0;
98
99      visit_children(&context, root);
100     return context.had_error;
101 }
```

Listing 3: Fungsi `validate_program`

- Konteks awal: kedalaman loop 0.
- Tipe kembalian awal `void`.
- `visit_children` menelusuri program.
- Mengembalikan status kesalahan.
- Driver menggabungkan hasil semua fase.

Klasifikasi Kesalahan

Kelas	Fase	Contoh
Leksikal	Lexer	karakter ilegal, string tidak ditutup
Sintaksis	Parser	titik koma hilang, kurung tidak seimbang
Semantik	Type checker	identifier tidak dideklarasikan, tipe salah
Validasi	Validator	`break` di luar loop, return hilang

Kesalahan *run-time* bukan kesalahan kompilasi. Guntoro membatasi jenis kesalahan program pada leksikal, sintaksis, dan semantik.

Contoh Kesalahan menurut Guntoro

Jenis	Contoh	Penjelasan
Leksikal	`while (a < b)`	kata kunci salah eja
Sintaksis	`a = x + (b * (c + d);`	kurung tidak seimbang
Semantik	`int siswa; siswa = "Yanuar";`	tipe string ke integer
Semantik	`b = b + 1;`	variabel belum dideklarasikan

Pesan menyebut baris, nama yang terlibat, dan jenis pelanggaran.

Pemulihan dan Batas Pelaporan

Strategi pemulihan

- lexer: lewati karakter ilegal;
- parser: sinkronisasi ke `;` atau `}`;
- type checker: `TYPE_ERROR` lalu lanjut;
- validator: catat dan lanjut ke simpul berikutnya.

Batas pelaporan

- Pemulihan agresif dapat menghasilkan pesan palsu.
- Setelah sejumlah kesalahan, berhenti dan beri ringkasan.
- Batas dapat dikonfigurasi, misalnya 20 kesalahan.
- Fokus pada akar masalah.

Pemulihan memungkinkan beberapa kesalahan dilaporkan dalam satu proses.

Format Diagnostik dan Kualitas Pesan

Format konsisten

`berkas:baris:kolom: error: pesan`

```
tests/invalid/break_outside.sc:2:5:
  error: break hanya valid di dalam loop
```

Listing 4: Contoh keluaran diagnostik

- buruk: `error`;
- cukup: `baris 4: type error`;
- baik: `baris 4, kolom 9: tidak dapat menetapkan float ke int; gunakan cast eksplisit.`

Contoh: `break` di Luar Loop

```
int main() {
    break;
    return 0;
}
```

Listing 5: Program tidak valid

- Validator memasuki fungsi dengan `loop_depth = 0`.
- Simpul `BREAK` diperiksa.
- Kedalaman nol: laporkan `break hanya valid di dalam loop`.
- Baris dan kolom menunjuk token `break`.

Contoh: Fungsi Tanpa `return`

```
int hitung(int x) {
    int y = x + 1;
}
```

Listing 6: Fungsi non-void tanpa return

- Tipe fungsi `int`; `has_return = 0`.
- Setelah anak dikunjungi, validator memeriksa flag.
- Pesan: `fungsi non-void harus mengembalikan nilai`.
- Lokasi menunjuk nama fungsi.

Kegiatan Praktikum: M5 Proyek Akhir

Langkah kerja

1. Integrasikan seluruh modul: lexer, parser, type checker, validator.
2. Tambahkan aturan validasi yang belum tercakup.
3. Uji dengan program valid dan tidak valid.
4. Ukur waktu dan penggunaan memori.
5. Tulis laporan dan siapkan demo.

Keluaran M5

CLI `scc`; pipeline lengkap; minimal 40 kasus uji; pesan kesalahan yang menyebut baris, nama, dan jenis pelanggaran; log debug; laporan dan demo.

Tugas dan Artefak

Artefak Sub-CPMK 4.4 dan 1.7

AST, validator kode, dan pesan kesalahan yang menyebut baris, nama, serta jenis pelanggaran, disertai keluaran pengujian.

- CLI memproses program valid tanpa kesalahan.
- CLI menolak program tidak valid dengan pesan jelas.
- Diagnostik menyebut baris, nama, dan jenis pelanggaran.
- Minimal 40 kasus uji valid dan tidak valid.
- Log *debug* dan perbaikan terdokumentasi.
- Laporan proyek dan demo selesai.

Pertemuan ini menutup increment proyek akhir; UAS menilai demonstrasi proyek dan ujian tulis.

Rangkuman dan Penutup Proyek

- AST yang baik ringkas, menyimpan lokasi, dan mudah ditelusuri.
- Traversal pre-order dan post-order melayani kebutuhan konteks dan sintesis.
- Validasi multi-pass memeriksa aturan kontrol setelah tipe.
- Kesalahan diklasifikasikan menurut fase dan dipulihkan secara terkendali.
- Diagnostik yang baik menyebut berkas, baris, kolom, nama, dan jenis pelanggaran.

Menuju UAS

Proyek akhir didemonstrasikan pada Pertemuan 16. Pastikan pipeline berjalan pada berkas uji, log debug lengkap, dan laporan siap.